POLYMORPHISM + ENCAPSULATION

Q1. What is polymorphism?
a) Many classes
b) Many forms of a method
c) One object many variables
d) Code duplication
Answer: b

Q2. How many types of polymorphism in Java?
a) 1
b) 2
c) 3
d) 4
Answer: b

Q3. Compile-time polymorphism is achieved by?
a) Overriding
b) Overloading
c) Inheritance
d) Abstraction
Answer: b

Q4. Runtime polymorphism is achieved by?
a) Overloading
b) Overriding
c) Constructors
d) Interfaces
Answer: b

Q5. Which binding happens in method overloading?
a) Runtime
b) Compile-time
c) Dynamic
d) Late
Answer: b

Q6. Which binding happens in method overriding?
a) Compile-time
b) Static
c) Runtime
d) Early
Answer: c

Q7. What will be output?

```
class A {
   void show() {
      System.out.println("A");
   }
}
class B extends A {
```

```
    void show() {
        System.out.println("B");
    }
}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        obj.show();
    }
}
```

a) A
b) B
c) Error
d) Null

Answer: b

Q8. What will be output?

```
class A {
    int x = 10;
}
class B extends A {
    int x = 20;
}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        System.out.println(obj.x);
    }
}
```

a) 10
b) 20
c) Error
d) Null

Answer: a

👉 Important: **Variables are not polymorphic**

Q9. Can constructors be polymorphic?

a) Yes
b) No
c) Only static
d) Only final

Answer: b

Q10. Which cannot be overridden?

a) public method
b) protected method
c) final method
d) abstract method

Answer: c

Q11. What will be output?

```
class A {
   static void show() {
      System.out.println("A");
   }
}
class B extends A {
   static void show() {
      System.out.println("B");
   }
}
public class Test {
   public static void main(String[] args) {
      A obj = new B();
      obj.show();
   }
}
```

a) A
b) B
c) Error
d) Null

Answer: a

👉 Static methods are NOT polymorphic

Q12. What is dynamic method dispatch?

a) Compile-time decision
b) Runtime method call resolution
c) Constructor call
d) Static binding

Answer: b

Q13. What will be output?

```
class A {
   void show() {
      System.out.println("A");
   }
}
class B extends A {
   void display() {
      System.out.println("B");
   }
}
public class Test {
   public static void main(String[] args) {
      A obj = new B();
      obj.display();
   }
}
```

a) A
b) B
c) Error
d) Null

Answer: c

Q14. Which concept allows one interface, many implementations?
a) Encapsulation
b) Polymorphism
c) Inheritance
d) Abstraction
Answer: b

Q15. Method overriding requires?
a) Same method name only
b) Same name + same parameters
c) Same return type only
d) Different parameters
Answer: b

- **Encapsulation**

Q16. What is encapsulation?
a) Binding data and methods together
b) Hiding code
c) Using inheritance
d) Looping
Answer: a

Q17. Encapsulation is achieved using?
a) Public variables
b) Private variables + getters/setters
c) Interfaces
d) Constructors only
Answer: b

Q18. Why use encapsulation?
a) Increase code length
b) Data hiding and security
c) Faster execution
d) Memory allocation
Answer: b

Q19. What is a getter method?
a) Updates value
b) Returns value
c) Deletes value
d) Initializes object
Answer: b

Q20. What is a setter method?
a) Returns value
b) Updates value
c) Deletes object
d) Calls constructor
Answer: b

Q21. What will be output?

```
class Test {
   private int x = 10;
   int getX() {
      return x;
   }
}
public class Main {
   public static void main(String[] args) {
      Test t = new Test();
      System.out.println(t.getX());
   }
}
```

a) 0
b) 10
c) Error
d) Null
Answer: b

Q22. What will happen?

```
class Test {
   private int x = 10;
}
public class Main {
   public static void main(String[] args) {
      Test t = new Test();
      System.out.println(t.x);
   }
}
```

a) 10
b) 0
c) Compile-time error
d) Runtime error
Answer: c

Q23. Which access modifier is best for encapsulation?

a) public
b) private
c) protected
d) default
Answer: b

Q24. Can we achieve encapsulation without methods?

a) Yes
b) No
c) Only static
d) Only final
Answer: b

Q25. What will be output?

```
class Test {
   private int x;
   void setX(int x) {
```

```
        this.x = x;
    }
    int getX() {
        return x;
    }
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.setX(50);
        System.out.println(t.getX());
    }
}
```

a) 0
b) 50
c) Error
d) Null
Answer: b

Q26. Encapsulation helps in?
a) Code duplication
b) Data hiding
c) Inheritance
d) Compilation
Answer: b

Q27. What is tightly encapsulated class?
a) All variables public
b) All variables private
c) No methods
d) Only static methods
Answer: b

Q28. What will happen?

```
class Test {
    private int x = 10;
    void setX(int x) {
        x = x;
    }
}
```

a) Value updated
b) No change in instance variable
c) Compile error
d) Runtime error
Answer: b
👉 Classic trap: missing this.x

Q29. Encapsulation improves?
a) Code readability
b) Data security
c) Maintainability

d) All of the above
Answer: d

Q30. What is combination of encapsulation + polymorphism used for?
a) Data duplication
b) Flexible and secure code design
c) Faster execution
d) Memory allocation
Answer: b